PROJETO FINAL

Loja Virtual E-commerce

Especificação organizada de requisitos e entrega

PESO DA AVALIAÇÃO

Este projeto corresponde a **80% da nota total do curso**. O foco deve ser a implementação correta dos requisitos essenciais, a compreensão da arquitetura e a capacidade de explicar as decisões tomadas, em especial as regras de carrinho, estoque e pedido.

Tecnologias principais: Laravel • PostgreSQL/MySQL • Docker • Swagger UI (openapi.yaml)

Sumário

SEÇÃO	CONTEÚDO
1	Visão geral e objetivo
2	Tecnologias e conceitos obrigatórios
3	Escopo funcional
4	Critérios de aceite por funcionalidade
5	Regras de negócio
6	Principais relacionamentos
7	Seeders
8	Dockerização
9	Extra pontuado: hospedagem
10	Checklist de entrega
11	Referência de rotas sugerida

> **PRIORIDADE RECOMENDADA**
> Conclua primeiro autenticação com papéis, catálogo, página de produto, carrinho sem login, fusão do carrinho no login, checkout com baixa de estoque, pagamento simulado, meus pedidos, área administrativa, seeders, Docker e Swagger UI. A hospedagem é o único extra pontuado e só deve ser feita depois que o essencial estiver funcionando. Se você fazer o extra e não saber explicar alguma coisa do básico, perderá pontos, em vez de ganhar.

1. Visão geral e objetivo

Objetivo principal. Desenvolver uma loja virtual completa, cobrindo a jornada de compra do visitante anônimo até o pedido pago. O frontend deve ser construído em HTML, CSS e JS **(se utilizar o blade, módulo frontend do Laravel, o projeto final será zerado)**, enquanto o backend deve ser uma API REST em Laravel organizada no padrão MVC + Service Layer.

Resultado esperado. Ao final, o sistema deve permitir que qualquer visitante navegue pelo catálogo e monte um carrinho sem estar logado; que esse carrinho seja preservado ao entrar na conta; que o usuário finalize o pedido com endereço de entrega, confirme um pagamento **simulado** e acompanhe o status; e que um administrador gerencie produtos, categorias, estoque e pedidos.

1.1 Escopo do projeto

- O foco está na **lógica de negócio**: catálogo, carrinho, fusão do carrinho no login, controle de estoque, criação do pedido e transição de status.
- O processamento de pagamento **não é real**. Deve ser simulado por uma ação explícita (por exemplo, um botão “Confirmar pagamento” / “Já está pago”) que altera o status do pedido.
- Não integrar gateway de pagamento real. Nenhum dado de cartão deve ser solicitado, trafegado ou armazenado em nenhum momento.
- Não é necessário cálculo real de frete. Um valor fixo ou uma regra simples (por exemplo, frete grátis acima de um valor) é suficiente, desde que documentada.
- O projeto deve demonstrar domínio dos fundamentos de JavaScript e integração completa com uma API Laravel.
- A interface deve ser funcional, responsiva (ou seja, tanto desktop quanto mobile) e coerente, mesmo que simplificada.

> **DICA IMPORTANTE**
> Não deixe no seu frontend elementos que não funcionam, apenas para parecer uma loja de verdade: avaliações fixas, “quem viu também comprou”, campo de cupom que não valida, frete por CEP que não calcula, filtros que não filtram. Uma aplicação estável e com todas as funcionalidades obrigatórias vale mais do que uma vitrine decorativa.

2. Tecnologias e conceitos obrigatórios

2.1 Backend - Laravel

- API REST responsável por autenticação, regras de negócio, persistência e respostas em JSON.
- **Padrão MVC + Service Layer**: Models para entidades e relacionamentos; Services para regras de negócio; Controllers para receber requisições e devolver respostas; Resources para saídas.
- As regras de carrinho, fusão de carrinhos, estoque e pedido devem viver na camada de **Service**, não dentro dos Controllers.
- Autenticação por sessão (por exemplo, Laravel Sanctum) com dois papéis: **cliente** e **admin**.
- Operações que alteram estoque e criam pedidos devem ser executadas dentro de uma **transação de banco**.

- Banco de dados PostgreSQL ou MySQL executado localmente pelo compose.yaml do backend.
- Seeders para popular o ambiente de desenvolvimento com dados de teste.
- Documentação interativa da API com Swagger UI.

3. Escopo funcional

MÓDULO	RESUMO
1. Autenticação	Registro, login, logout, manutenção de sessão e papéis (cliente e admin).
2. Catálogo	Listagem pública de produtos com busca, filtro por categoria, ordenação e paginação.
3. Produto	Página de detalhe com imagem, descrição, preço, disponibilidade e adicionar ao carrinho.
4. Carrinho sem login	Carrinho funcional para visitantes, persistido no servidor e identificado por sessão.
5. Carrinho após login	Carrinho vinculado à conta e fusão do carrinho de visitante ao entrar.
6. Checkout	Endereço de entrega, revisão dos itens, cálculo do total e criação do pedido.
7. Pagamento simulado	Ação explícita que marca o pedido como pago. **Sem gateway real**.
8. Meus pedidos	Histórico e detalhe do pedido com itens, valores praticados e status.
9. Administração	CRUD de produtos e categorias, controle de estoque e gestão dos status dos pedidos.
10. Perfil	Dados do usuário e gerenciamento de endereços.
11. Navegação	Catálogo, busca, carrinho, meus pedidos, perfil e, para o admin, o painel.
12. Extra	Hospedagem pública, permitindo registro e compras (fictícias) por usuários reais.

4. Critérios de aceite por funcionalidade

4.1 Registro, login e papéis

- ☐ O usuário consegue se registrar com nome, e-mail e senha.
- ☐ O sistema valida campos obrigatórios, e-mail único e confirmação de senha, informando erros de forma legível.
- ☐ O usuário consegue fazer login com credenciais válidas e recebe um cookie de acesso.
- ☐ Existe ação de logout que invalida a sessão.
- ☐ A sessão permanece ativa conforme a estratégia de autenticação escolhida.
- ☐ Cada usuário possui um papel (cliente ou admin) e a navegação do frontend reflete esse papel.
- ☐ Um cliente que tenta acessar rotas administrativas recebe 403.

4.2 Catálogo

- ☐ Visitantes **não autenticados** conseguem ver a listagem de produtos.
- ☐ Cada produto exibe imagem, nome, preço e indicação de indisponibilidade quando o estoque é zero.
- ☐ Permite pesquisar produtos por nome.
- ☐ Permite filtrar por categoria.
- ☐ Permite ordenar por preço (crescente e decrescente) e por mais recentes.
- ☐ A listagem é paginada e a paginação funciona no frontend.
- ☐ Produtos inativos não aparecem no catálogo.
- ☐ Uma busca sem resultados exibe mensagem apropriada, e não uma tela vazia sem explicação.

4.3 Página do produto

- ☐ É acessível ao clicar no produto e possui URL própria.
- ☐ Exibe nome, descrição, preço, categoria, imagem e disponibilidade.
- ☐ Permite escolher a quantidade, limitada ao estoque disponível.
- ☐ Permite adicionar ao carrinho, com ou sem login.
- ☐ Produto sem estoque não pode ser adicionado, e a interface deixa isso claro.
- ☐ Produto inexistente ou inativo retorna 404.

4.4 Carrinho sem login (visitante)

- ☐ O visitante consegue adicionar itens ao carrinho **sem estar autenticado**.
- ☐ O carrinho é identificado pela sessão, gerada pela API e guardado pelo frontend.
- ☐ O carrinho do visitante é persistido no banco e sobrevive a recarregar a página e a fechar o navegador.
- ☐ Permite alterar a quantidade e remover itens.
- ☐ Exibe subtotal por item e total do carrinho, sempre calculados pelo servidor.
- ☐ Adicionar um produto já presente no carrinho soma a quantidade, sem duplicar a linha.

- ☐ Ao tentar finalizar a compra, o visitante é direcionado ao login ou registro **sem perder o carrinho**.

4.5 Carrinho após o login

- ☐ Ao autenticar, o carrinho de visitante é **fundido** com o carrinho da conta.
- ☐ A regra de fusão está implementada e o aluno sabe explicá-la (ver seção 5).
- ☐ Após a fusão, a sessão inicial de visitante deixa de valer e não gera um segundo carrinho.
- ☐ Um usuário autenticado tem no máximo um carrinho ativo.
- ☐ O carrinho da conta persiste entre sessões e entre dispositivos.
- ☐ Itens indisponíveis (produto inativo ou sem estoque) são sinalizados no carrinho.
- ☐ O total é sempre recalculado no servidor; valores enviados pelo cliente são ignorados.

4.6 Checkout

- ☐ Exige usuário autenticado.
- ☐ Permite cadastrar ou selecionar um endereço de entrega.
- ☐ Exibe o resumo do pedido: itens, quantidades, preços unitários, frete e total.
- ☐ Carrinho vazio não permite finalizar, com erro claro (422).
- ☐ Item sem estoque suficiente impede a finalização e informa qual produto falhou.
- ☐ Ao confirmar, é criado um pedido com status “aguardando pagamento”.
- ☐ O estoque dos produtos é decrementado na criação do pedido.
- ☐ O carrinho é esvaziado após a criação do pedido.
- ☐ Os preços do pedido ficam congelados: alterações posteriores no catálogo não alteram pedidos já criados.

4.7 Pagamento simulado

- ☐ A tela do pedido pendente exibe uma ação explícita de pagamento (por exemplo, “Confirmar pagamento”).
- ☐ Ao acionar, o pedido passa para “pago” e registra a data e hora do pagamento.
- ☐ Nenhum dado de cartão é solicitado, trafegado ou armazenado.
- ☐ Somente o dono do pedido pode pagá-lo; outro usuário recebe 403.
- ☐ Pagar um pedido que não está aguardando pagamento retorna erro coerente (409).
- ☐ A interface deixa claro que o pagamento é simulado para fins acadêmicos.

4.8 Meus pedidos

- ☐ Lista os pedidos do usuário com identificador, data, total e status.
- ☐ Permite abrir o detalhe do pedido.
- ☐ O detalhe exibe itens, quantidades, preços praticados, endereço de entrega e status atual.
- ☐ O usuário só enxerga os próprios pedidos; acessar pedido alheio retorna 403 ou 404.
- ☐ O cliente consegue cancelar um pedido enquanto ele estiver aguardando pagamento, devolvendo o estoque.

4.9 Área administrativa

- ☐ Acessível somente a usuários com papel admin.
- ☐ Permite criar, editar e excluir produtos (nome, descrição, preço, estoque, categoria, imagem e ativo/inativo).
- ☐ Permite criar, editar e excluir categorias.
- ☐ Impede excluir categoria com produtos vinculados, ou adota outro comportamento claramente documentado.
- ☐ Permite atualizar o estoque de um produto.
- ☐ Lista todos os pedidos da loja, com filtro por status.
- ☐ Permite avançar o status do pedido (pago → enviado → entregue), respeitando as transições válidas.
- ☐ Excluir ou desativar um produto não quebra pedidos antigos (ver seção 5).

4.10 Perfil e navegação

- ☐ O usuário visualiza e edita seus dados e gerencia seus endereços.
- ☐ A navegação principal dá acesso a Catálogo, Carrinho, Meus pedidos e Perfil; o admin também acessa o Painel.
- ☐ O indicador do carrinho reflete a quantidade de itens, com e sem login.
- ☐ O item ativo da navegação é visualmente identificado.
- ☐ Rotas inexistentes possuem tratamento adequado, como redirecionamento ou página 404.
- ☐ Rotas protegidas acessadas sem autenticação redirecionam para o login.

5. Regras de negócio

Acesso e autorização

- Catálogo, detalhe do produto e carrinho de visitante são **públicos**. Checkout, pedidos, perfil e administração exigem autenticação.
- Somente o admin cria, edita ou exclui produtos e categorias e altera o status dos pedidos.
- Somente o dono do pedido, ou um admin, pode visualizá-lo, pagá-lo ou cancelá-lo.

Carrinho

- Um usuário autenticado possui no máximo **um carrinho ativo**.
- A combinação carrinho + produto é única; adicionar um item já existente soma a quantidade.
- A quantidade mínima é 1 e a quantidade 0 remove o item.
- A quantidade não pode ultrapassar o estoque disponível.
- Produto inativo ou sem estoque não pode ser adicionado ao carrinho.
- O carrinho de visitante é identificado por sessão, que não pode ser adivinhado nem reutilizado por outro visitante.
- O carrinho **não congela preços**: ele sempre reflete o preço atual do produto. O congelamento acontece apenas no pedido.

Fusão de carrinhos no login - regra obrigatória

Esta é a regra central do projeto e o aluno deve saber explicá-la na avaliação.

- Ao autenticar (login ou registro), se o frontend enviar uma sessão que possui carrinho de visitante, a API deve fundir esse carrinho ao carrinho da conta (vazio ou não).
- Produto presente **apenas** no carrinho de visitante: é copiado para o carrinho do usuário.
- Produto presente **nos dois** carrinhos: as quantidades são somadas, limitadas ao estoque disponível.
- Se a quantidade final exceder o estoque, ela é ajustada para o máximo disponível e o usuário é avisado.
- Concluída a fusão, o carrinho de visitante é descartado e sua sessão é invalidada.
- Outra regra de fusão (por exemplo, prevalecer a maior quantidade) é aceita, desde que esteja documentada no README e implementada de forma consistente.

Pedido, estoque e pagamento

- O total do pedido é sempre calculado no servidor, a partir dos preços gravados no banco.
- Ao criar o pedido, cada item guarda o **preço unitário e o nome do produto** praticados naquele momento.
- A criação do pedido valida o estoque e o decrementa dentro de uma transação: se qualquer item falhar, nada é gravado.
- Status possíveis: aguardando pagamento → pago → enviado → entregue; e cancelado a partir de aguardando pagamento.
- Somente transições válidas são aceitas; uma transição inválida retorna 409.
- O cancelamento devolve o estoque dos itens do pedido.
- O pagamento é **simulado**: nenhuma integração real e nenhum dado de cartão.

- Valores monetários usam decimal(10,2) no banco ou inteiros em centavos — **nunca ponto flutuante** — e a escolha deve ser consistente em toda a aplicação.
- Produtos que já pertencem a pedidos não devem ser removidos fisicamente: use exclusão lógica (ativo/inativo ou soft delete) para preservar o histórico.
- Respostas devem utilizar códigos HTTP coerentes e um formato de erro consistente.

6. Principais relacionamentos

RELAÇÃO	DESCRIÇÃO
User 1:N Order	Um usuário pode ter vários pedidos.
User 1:1 Cart	Um usuário autenticado tem no máximo um carrinho ativo. O carrinho de visitante tem user_id nulo.
Cart 1:N CartItem	Um carrinho contém vários itens.
Product 1:N CartItem	Um produto pode estar em vários carrinhos; a dupla carrinho + produto é única.
Category 1:N Product	Uma categoria agrupa vários produtos.
Order 1:N OrderItem	Um pedido contém vários itens, com preço e nome congelados.
Product 1:N OrderItem	Um produto pode aparecer em vários pedidos.
User 1:N Address	Um usuário pode ter vários endereços; o pedido guarda o endereço utilizado.
Order 1:1 Payment	Registro do pagamento simulado: status, valor e data/hora. Pode ser uma tabela própria ou campos no pedido.

7. Seeders

A API deve possuir seeders que populem o banco de desenvolvimento com dados suficientes para verificar **todas** as regras implementadas, incluindo:

- Um usuário admin e ao menos três clientes, com credenciais documentadas no README.
- Ao menos três categorias.
- Ao menos trinta produtos distribuídos entre as categorias, com preços e estoques variados.
- Ao menos um produto com estoque zero e um produto inativo, para permitir testar as regras de indisponibilidade.
- Pelo menos um carrinho de visitante e um carrinho de usuário já populados.
- Pedidos em status diferentes: aguardando pagamento, pago, enviado, entregue e cancelado.

8. Dockerização

8.1 Frontend

- Deve existir somente um Dockerfile para o frontend.
- O Dockerfile deve instalar dependências e executar/buildar o projeto de forma reproduzível.
- A URL da API deve ser configurável por variável de ambiente.
- Arquivos e pastas desnecessários devem ser ignorados por .dockerignore.

8.2 Backend

- Dockerfile.dev para desenvolvimento.
- Dockerfile para produção.
- Volume persistente para o banco de dados.
- Volume ou storage configurado para as imagens de produtos, de modo que os uploads sobrevivam à recriação do container.
- Variáveis de ambiente para conexão com o banco, URL da aplicação e segredos.

8.3 Compose

- Apenas um compose.yaml para todo o projeto, contendo os serviços do frontend, api e banco de dados.

9. Extra pontuado: hospedagem

A hospedagem é o **único extra pontuado** deste projeto.

EXTRA	PONTUAÇÃO	CONDIÇÃO
Aplicação hospedada	+10 pontos	Frontend e backend acessíveis publicamente, permitindo registro e compra por usuários reais.

9.1 Critérios de aceite da hospedagem

- ☐ Frontend e backend acessíveis publicamente.
- ☐ Banco de dados de produção configurado.
- ☐ Uploads e imagens de produtos persistentes e acessíveis.
- ☐ CORS e URLs configurados para o domínio real.
- ☐ Migrations executadas em produção.
- ☐ Registro habilitado para novos usuários.
- ☐ Catálogo com produtos na produção, via seeders ou cadastro pelo painel administrativo.
- ☐ Aplicação testada em janela anônima e em outro dispositivo, percorrendo o fluxo carrinho sem login → login → pedido pago.

SOBRE OUTROS RECURSOS

Não há outros extras pontuados neste projeto. Recursos como cupom de desconto, avaliações de produto, lista de desejos ou frete por CEP são livres, mas **não acrescentam pontos** e não devem comprometer os requisitos obrigatórios.

10. Checklist de entrega

10.1 Funcionalidades obrigatórias

- ☐ Registro, login e logout funcionando, com papéis cliente e admin.
- ☐ Catálogo público com busca, filtro por categoria, ordenação e paginação.
- ☐ Página de produto com detalhes e ação de adicionar ao carrinho.
- ☐ Carrinho funcional sem login, persistido no servidor por sessão.
- ☐ Fusão do carrinho de visitante com o carrinho da conta no login.
- ☐ Alterar quantidade e remover itens do carrinho, com totais calculados no servidor.
- ☐ Checkout com endereço de entrega e criação do pedido.
- ☐ Baixa de estoque na criação do pedido e devolução no cancelamento.
- ☐ Pagamento simulado alterando o status do pedido.
- ☐ Meus pedidos com listagem e tela de detalhe.
- ☐ Área administrativa com CRUD de produtos e categorias e gestão dos status dos pedidos.
- ☐ Navegação com catálogo, carrinho, meus pedidos, perfil e painel do admin.
- ☐ Não há elementos visuais sem funcionamento apenas para imitar uma loja real.

10.2 Backend e banco de dados

- ☐ API Laravel organizada no padrão MVC + Service Layer.
- ☐ Regras de carrinho, estoque e pedido implementadas na camada de Service.
- ☐ Migrations e relacionamentos principais configurados.
- ☐ Autenticação e autorização aplicadas, incluindo o bloqueio das rotas administrativas.
- ☐ Unicidade de carrinho + produto e limites de estoque respeitados.
- ☐ Criação do pedido executada dentro de uma transação.
- ☐ Preços congelados nos itens do pedido.
- ☐ Seeders populam o banco de desenvolvimento.
- ☐ Banco PostgreSQL ou MySQL funcionando.
- ☐ Respostas e códigos HTTP consistentes.

10.3 Docker, Swagger UI e repositório

- ☐ Frontend possui um único Dockerfile.
- ☐ Backend possui Dockerfile.dev, Dockerfile de produção.
- ☐ Projeto tem um compose.yaml que sobe todos os serviços necessários.
- ☐ Swagger UI está acessível e permite consultar a documentação da API.
- ☐ Repositório no GitHub está acessível pelo link enviado.
- ☐ README contém as instruções de execução, as credenciais dos seeders e a regra de fusão de carrinho adotada.
- ☐ .env real não foi versionado e existe um .env.example sem segredos reais.

10.4 Extra opcional

- ☐ Aplicação hospedada com registro e compra por usuários reais: **+10 pontos**.

10.5 Conferência final

> **O PROJETO ESTÁ PRONTO QUANDO...**
> O link do GitHub está acessível, o README permite subir o frontend e o backend, os seeders populam o banco e é possível percorrer, sem nenhum ajuste manual no código, o fluxo completo: navegar no catálogo sem login, montar um carrinho, entrar na conta e ver o carrinho preservado, finalizar o pedido com endereço, confirmar o pagamento simulado e acompanhar o status do pedido — inclusive pelo painel do administrador.

Foco: completar bem o essencial, manter o projeto funcional e saber explicar as decisões tomadas.

11. Referência de rotas sugerida

Esta seção é uma **sugestão**, não uma obrigação. Outra organização de rotas é aceita, desde que esteja documentada no Swagger UI e siga os mesmos comportamentos.

ROTA	DESCRIÇÃO	ACESSO
POST /api/auth/register	Cria a conta. Aceita a sessão do carrinho de visitante para a fusão.	Público
POST /api/auth/login	Autentica. Aceita a sessão do carrinho de visitante para a fusão.	Público
POST /api/auth/logout	Encerra a sessão e invalida a sessão.	Autenticado
GET /api/products	Lista com busca, filtro por categoria, ordenação e paginação.	Público
GET /api/products/{id}	Detalhe do produto.	Público
GET /api/categories	Lista as categorias.	Público
GET /api/cart	Retorna o carrinho atual e os totais.	Público
POST /api/cart/items	Adiciona um item ao carrinho de visitante ou do usuário.	Público
PATCH /api/cart/items/{id}	Altera a quantidade de um item.	Público
DELETE /api/cart/items/{id}	Remove um item do carrinho.	Público
GET / POST /api/addresses	Lista e cadastra endereços do usuário.	Autenticado
POST /api/orders	Cria o pedido a partir do carrinho e baixa o estoque.	Autenticado
GET /api/orders	Lista os pedidos do usuário.	Autenticado
GET /api/orders/{id}	Detalhe do pedido.	Dono ou admin
POST /api/orders/{id}/pay	Pagamento simulado: marca o pedido como pago.	Dono do pedido
POST /api/orders/{id}/cancel	Cancela o pedido e devolve o estoque.	Dono ou admin
POST / PUT / DELETE /api/admin/products	CRUD de produtos.	Admin
POST / PUT / DELETE /api/admin/categories	CRUD de categorias.	Admin

ROTA	DESCRIÇÃO	ACESSO
GET /api/admin/orders	Lista todos os pedidos da loja.	Admin
PATCH /api/admin/orders/{id}/status	Altera o status do pedido.	Admin

Laravel • API REST • Docker • Swagger UI